

Sohan Mandal

Roll 2405912 · GitHub `sohan1611` · CSE-14 · Reviewer: Sohini Pandit (Sprint A), Sinjan Mishra (Sprint B)

You own the network and the evidence. First, the channel emulator every teammate tests against — which the catalogue requires us to write ourselves, so the AI draft must be replaced by your own code. Then the machinery that turns working protocols into marks: per-run statistics, the experiment runner, plots with confidence intervals, and the proof that our emulator tells the truth. You are also the only one of us with Claude Code, which makes you the group's best debugging resource — for helping teammates understand their own code, never for writing it.

Thu 17 Sep

Emulator rewrite merged — Shaili cannot test Stop-and-Wait without it

Tue 22 Sep

Configs locked, runtime dry run, file-size decision, fallback call

Sun 27 Sep

Figures 1–3 generated by script — the sprint's last gate

YOU DEPEND ON	DEPEND ON YOU
Shaili — the sender's <code>RESULT</code> line on Sun 20 All three protocols — merged by Thu 24 for the full runs	Shaili — the emulator on Thu 17 Sinjan — trace lines with <code>type</code> and <code>ack</code> , for Experiment 4 Sohini — the emulator for Selective Repeat; the Tue 22 decision Everyone — the harness and every figure

Contents

1 · Get set up	9 · Experiment 3
2 · Your part of the system	10 · Your report section
3 · Concepts you must master	11 · How we work together
4 · Design decisions you must defend	12 · Using AI without losing marks
5 · Your day-by-day plan	13 · Viva preparation
6 · Team conventions	14 · Help, references, glossary
7 · How to test your work	15 · Sprint checklist
8 · When something breaks	

1 · Get set up — by Tuesday 15 September, 10 pm

Gate 1 is the first check of the sprint: all four of us have cloned the repository, verified our git identity, and pushed a first commit. Everything after it depends on this.

The repository is not a shared folder. Sohan's copy lives on Sohan's laptop. You make your own complete copy on *your* laptop by cloning it from GitHub, then you work, commit and push from there. GitHub is only the meeting point. Every command in this handbook runs on your own machine. (If you already have a working clone, start at step 4.)

Step 1 — Install the tools

- **Git.** On Windows this also installs **Git Bash** — run every command in this handbook there, not in PowerShell or Command Prompt.
- **JDK 21** (Adoptium Temurin). Check that `java -version` and `javac -version` both report 21.
- **WSL2 with Ubuntu** on Windows, because the experiments run on Linux. Inside Ubuntu install the JDK and Python too:

```
wsl --install -d Ubuntu
wsl -d Ubuntu
sudo apt update && sudo apt install -y openjdk-21-jdk make python3-pip
java -version
```

Step 2 — Clone onto your laptop

```
git clone https://github.com/sohan1611/rdt-udp.git
cd rdt-udp
```

You now have your own `rdt-udp` folder. Everything from here on happens inside it.

Step 3 — Tell git who you are, inside that folder

```
git config user.name "Sohan Mandal"
git config user.email "the-email-on-your-github-account"
```

This email decides whether you count as a contributor. It must be one of the addresses listed at github.com/settings/emails for your account `sohan1611`. If you would rather not use a personal address, that page shows a private one ending in `@users.noreply.github.com` that links just as well. Commits with an unrecognised email show a grey avatar and are credited to nobody — and sir treats commit history as evidence of individual contribution.

Step 4 — Confirm the build

```
./build.sh
```

Every suite must report **0 failed** — 54 tests across four suites as of 16 September. If anything fails, paste the output in the group chat the same evening.

Step 5 — Push your first commit

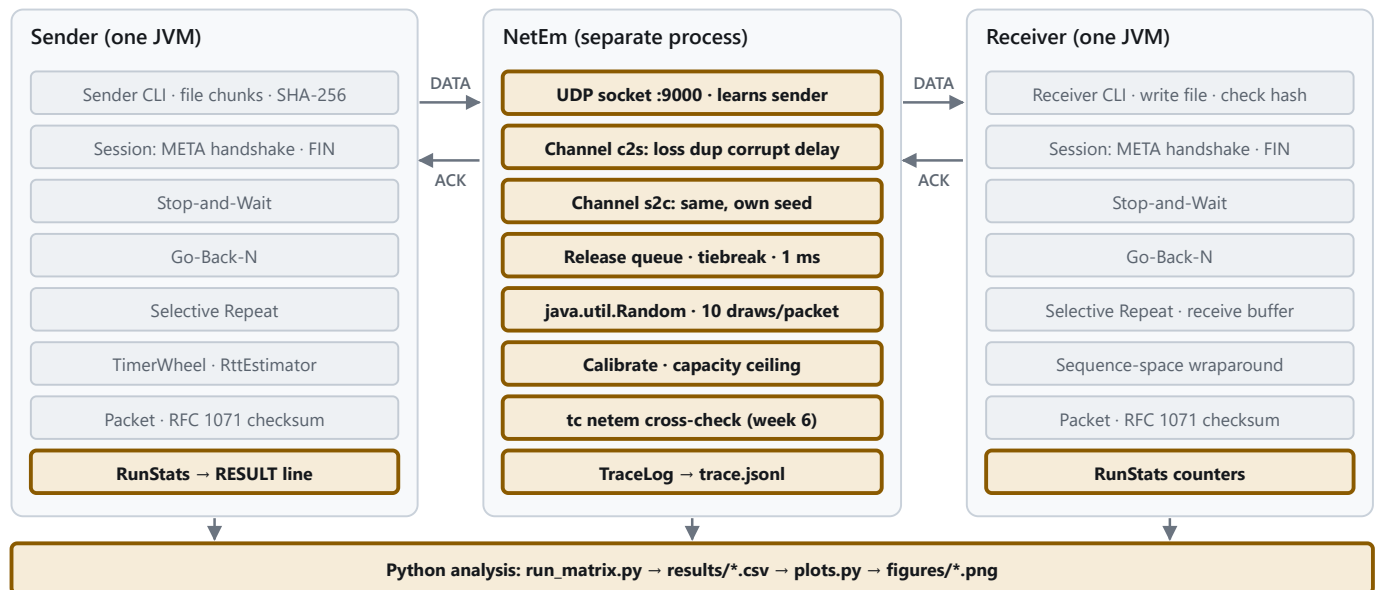
Open `AI-USE.md` and replace `_[name]_` beside your role heading with your name. Then:

```
git checkout -b setup/sohan
git add AI-USE.md
git commit -m "Add Sohan to AI-USE.md"
git push -u origin setup/sohan
git log -1 --format='%an <%ae>'
```

The last command must print your own name and your GitHub email. Open the pull request on GitHub and ask Sohini Pandit to approve it. Your first push asks you to sign in to GitHub; on Windows a browser window opens, which is normal.

Done when: your PR is merged, `git log` shows your own name and email, and you can see yourself on the repository's Contributors page.

2 · Your part of the system



Bold, coloured boxes are yours. Grey boxes belong to teammates — either you depend on them, or they depend on you. Packets always travel through NetEm: the sender and receiver never talk directly.

Files you own

FILE	STATUS	WHAT IT DOES
<code>src/main/java/emulator/ChannelConfig.java</code> , <code>TraceLog.java</code>	AI draft — rewrite, Mon 14	Parses impairment specs; writes the JSONL decision trace
<code>src/main/java/emulator/Channel.java</code>	AI draft — rewrite, Tue 15 – Wed 16	The impairment model: decides what happens to each datagram
<code>src/main/java/emulator/NetEm.java</code>	AI draft — rewrite, Wed 16 – Thu 17	The middlebox process: socket loop and release queue
<code>src/main/java/emulator/Calibrate.java</code>	AI draft — rewrite, Fri 18	Measures the sustainable packet rate on the experiment machine
<code>src/main/java/rdt/RunStats.java</code>	New, Sat 19	Per-run counters, printed as the RESULT line (conventions 8–10)
<code>analysis/run_matrix.py</code>	New, Mon 21	Runs every cell × seed and appends rows to <code>results/*.csv</code>
<code>analysis/configs/</code>	New, Tue 22	One locked config per experiment
<code>analysis/plots.py</code>	New, Wed 23	Every figure, with 95% confidence intervals, from committed CSVs
<code>analysis/spurious.py</code>	Week 6, with Sinjan	Labels retransmissions for Experiment 4
<code>analysis/netem_validate.sh</code>	Buffer week / week 6	The <code>tc netem</code> cross-validation run
<code>src/test/java/rdt/ChannelTest.java</code> , <code>NetEmSmokeTest.java</code>	Exist, 20 tests — keep	Your specification for the rewrite

3 · Concepts you must master

3.1 · How the middlebox works

- **One socket, one thread.** NetEm listens on one UDP port. The first datagram from any address other than the receiver identifies the sender — which is why the sender must speak first, and why it may use any port.
- **Two independent channels.** `c2s` impairs sender-to-receiver traffic, `s2c` impairs the ACKs coming back, each with its own seeded generator, so loss on data never disturbs the random sequence for ACKs.
- **A release queue.** Every surviving copy is scheduled for a release time and held in a `PriorityQueue`. The loop sends everything due, then blocks in `receive` for exactly as long as the next release is away — at least 1 ms, because a timeout of 0 means forever, and at most 200 ms, so a stop request is noticed promptly.
- **A tiebreak.** `PriorityQueue` gives no order to equal keys. Without an insertion counter, two packets with the same release time could leave in either order — reordering the model never asked for, and a run that stops being reproducible.

3.2 · The impairment model, and why exactly 10 random draws

For each datagram the channel takes **ten** random numbers, always, in a fixed order: loss; duplicate; corrupt; corrupt position; jitter; reorder; then corrupt, corrupt position, jitter and reorder again for a possible duplicate copy. Only then does it decide: drop it, or build the primary copy (maybe flip one bit; delay = base + uniform jitter, clamped at zero; plus the reorder penalty if chosen), and build a second copy if duplicated.

Why take ten even when the packet is dropped? Because then changing the loss rate does not shift any other decision. Two sweep cells that share a seed see the *same* delay and jitter for every surviving packet, so the difference between their results is caused by the variable under test and nothing else. This is the **common random numbers** technique, and it visibly reduces the noise between neighbouring points on a curve. `ChannelTest` guards it: "changing loss does not disturb the delay sequence". **If you add an impairment, add its draws at the end and raise the constant** — never draw conditionally.

3.3 · Reproducibility and seeds

- `java.util.Random` has its algorithm written into the Java specification — a 48-bit linear congruential generator — so the same seed produces the same numbers on every JVM and machine. `ThreadLocalRandom` and `SecureRandom` make no such promise.
- **Do not pass seeds 1 to 10 straight through.** `java.util.Random` only lightly scrambles its seed before the first step, so seeds that differ by one give noticeably similar first outputs. Have `run_matrix.py` derive each run's seed from its name — for instance the first 8 bytes of SHA-256 of `exp1-sr-loss5-seed3`, masked below 2^{63} so Java's `Long.parseLong` accepts it. Same name, same seed, well spread.
- **The second direction's seed.** The draft uses `seed XOR 0x5DEECE66D` — the very constant `Random` uses internally. It works, but you will be asked why that number. Keep it and explain it, or derive the second seed with `new Random(seed).nextLong()`, which mixes the bits and is easy to justify. Either way, keep a test proving the two directions produce different decisions.

3.4 · Corruption, delay, reordering

- **One bit flipped, anywhere in the datagram.** Shaili's checksum catches every single-bit error, so "a corrupted packet never reaches the application" is a guarantee we can test. Real links corrupt in bursts; we choose the model that makes the invariant checkable, and the report should say so.
- **Reordering** holds a packet back by `reorderExtra`, 20 ms by default, so the next one overtakes it. That only works if packets are closer together than 20 ms. A window of 32 over a 40 ms round trip spaces them about 1.25 ms apart — reordering happens. Stop-and-Wait has one packet in flight, so *nothing can overtake it*, which is the key prediction for Experiment 3.

3.5 · The trace, and the change it needs

One JSON line per decision, flushed every 256 lines, so a run killed mid-way loses at most 256. **In your rewrite, add type and ack to every line** (convention 11), read from bytes 1 and 8–11. Today the emulator reads only the `seq` field, and every ACK is built with `seq 0`, so all ACKs look identical — and Experiment 4 must know which ACK was lost. The rewrite is the cheapest moment to fix it.

When a packet is duplicated, the trace holds an extra line with action `dup` or `dup+corrupt`. That line is a copy the emulator made, not a transmission — analysis that counts transmissions must skip it.

3.6 · Calibration

A perfect channel, offered traffic at stepped rates, keeping the highest rate that still delivers 99% of packets. On Windows the ceiling was **64,000 packets per second**, so the experiment budget is half that, 32,000. Now compare the load our experiments actually generate over a 40 ms round trip:

Stop-and-Wait	1 packet per round trip	≈	25 packets/s	
window 32	32 per round trip	≈	800 packets/s	
window 128	128 per round trip	≈	3,200 packets/s	– ten times under budget

The experiments run in WSL2, so the figure that counts is a WSL2 measurement. Java is not yet installed inside your Ubuntu — checked on 14 September — so installing it comes first on Friday 18.

3.7 · Measuring honestly

- **One transfer at a time.** Parallel runs compete for CPU and distort each other's timings.
- **Run from inside WSL's own filesystem** — clone into `~/rdt-udp`, not `/mnt/d/...`. File access through `/mnt` is far slower and adds noise to every transfer.
- **JIT warm-up.** Each run starts fresh JVMs, which begin slow. On Tue 22, run one configuration ten times and check whether goodput varies more than its confidence interval. If not, document that and move on; if so, add a discarded warm-up transfer.
- **Heap and GC.** Run with `-Xms512m -Xmx512m` and `-Xlog:gc:file=...`. Only claim "no collection during timed runs" if the logs show it.
- **Failures are data.** A run that times out writes a row marked failed. It is never silently skipped, so gaps in the results are visible.
- **Resumable.** The matrix takes hours. `run_matrix.py` skips cells already in the CSV, so a crash does not mean starting again.

3.8 · Confidence intervals

```
for each cell:  n runs,  mean  $\bar{x}$ ,  sample standard deviation  $s$ 
95% CI  =   $\bar{x} \pm t \times s / \sqrt{n}$ 

n = 3      t = 4.303
n = 5      t = 2.776
n = 10     t = 2.262
```

Write the three t values into your code rather than adding a statistics library. Note the first row: with 3 seeds the error bars are almost twice as wide as with 10. That is why Sunday 27's figures are a first pass, and why the buffer week reruns them at 10 seeds. Every figure states its n .

3.9 · Runtime — Stop-and-Wait dominates

FILE SIZE	ONE S&W RUN, 0% LOSS	ONE S&W RUN, 20% LOSS	EXP 1 S&W CELLS, 1 SEED	... 10 SEEDS
8 MiB	4.0 min	10.7 min	44 min	7.3 h
2 MiB	1.0 min	2.7 min	11 min	1.8 h

Estimates assuming loss in both directions and an RTO near 120 ms. Experiment 3's Stop-and-Wait cells add roughly half as much again.

So on Tuesday 22 you run **one seed of every cell** as a dry run, time it, and decide: 8 MiB or 2 MiB for the *whole* experiment, every protocol. Goodput is a rate, so the file size changes the running time, not the quantity measured — but the report states the choice.

3.10 · How `run_matrix.py` should work

1. Read the experiment's config: its fixed settings, the variable and its levels, the protocols, the seed count.
2. For every cell and seed: choose free ports; derive the seed (3.3); start NetEm with a trace file; start the receiver; start the sender; wait, with a timeout.

- 3. Take the sender's single `RESULT` line, parse its JSON, add the run settings, append one CSV row. On a timeout, append a failed row instead.
- 4. Stop NetEm cleanly so its trace is closed and flushed.

Config format. `docs/PLAN.md` says YAML, which needs PyYAML installed. TOML, read with Python's built-in `tomllib`, or plain JSON, needs nothing. Decide on Tuesday 22 and stick to it.

3.11 · Cross-checking against Linux `tc netem`

Your WSL2 kernel builds netem as a loadable module (`CONFIG_NET_SCH_NETEM=m`) and `tc` is installed — checked on 14 September. The sender talks to the receiver directly on loopback, with Linux doing the impairment instead of our emulator:

```
sudo modprobe sch_netem
sudo tc qdisc add dev lo root netem delay 20ms loss 5%
# run the receiver on 9001 and point the sender straight at 127.0.0.1:9001 – no NetEm
sudo tc qdisc del dev lo root
```

Always remove the qdisc, even after a failed run. While it is active, *everything* on that machine's loopback is delayed and lossy. Put the `del` command in a `trap` in the script so it runs whatever happens.

On loopback each packet passes the qdisc once, so data and ACKs each get the delay and loss — the same as our `--both`. Expect some differences: netem's jitter reorders packets by default, and its delay distribution differs from our uniform one. Report them honestly. **Try one run in the buffer week, not week 6**, so any kernel surprise has time to be solved.

4 · Design decisions you must defend

DECISION	WHY
A separate process	The protocols contain no test-only code and genuinely do not know they are being measured; it works across machines; it swaps for <code>tc netem</code> with no code change.
<code>java.util.Random</code> , seeds derived by name	Specified algorithm, reproducible anywhere; derived seeds avoid its weak first outputs for nearby seeds.
Exactly 10 draws per packet	Common random numbers — sweep cells differ only in the variable under test.
Single-bit corruption	Guaranteed detectable, so "corrupt data never reaches the application" is testable.
Timing accurate to about 1 ms	Under a few percent of our 20 ms delays, and identical for every protocol, so it cannot bias a comparison.
<code>type</code> and <code>ack</code> in the trace	Without them ACKs are indistinguishable and Experiment 4 cannot label anything.
One run at a time; failures recorded	No CPU contention; missing data is visible, not hidden.
A published capacity ceiling	Proves every data point measures the protocol, not a saturated machine.
Every figure generated by script	Anyone can regenerate the report's evidence from committed CSVs.

5 · Your day-by-day plan

Sprint A · 14–20 September

Mon 14 – Thu 17

Rewrite the emulator by hand · BLOCKS SHAILI

1. `git checkout -b m4/emulator-rewrite`. Read each draft file once for its shape, then write your own.
2. **Mon 14:** `ChannelConfig` and `TraceLog`. Add `type` and `ack` to trace lines now.
3. **Tue 15 – Wed 16:** `Channel`, from 3.2. Get every `ChannelTest` case green — the determinism and common-random-numbers tests are the ones that matter most.
4. **Wed 16 – Thu 17:** `NetEm`, from 3.1. `NetEmSmokeTest` tells you when it works.
5. Add a test that an ACK's trace line records its type and ack number, and that a duplicate's extra line is marked `dup`.
6. PR by Thursday afternoon. Sohini reviews it with three *why* questions. Merge by Thursday night.

Done when: `ChannelTest` 14/14 and `NetEmSmokeTest` 6/6 pass on code you wrote, merged Thursday night.

Be able to explain: questions 1, 2, 3 and 4 in section 13.

Fri 18

Calibrate on the real host · conventions call

1. Rewrite `Calibrate` by hand — it is on the same `AI-USE.md` open item as the emulator. Tick that box when both are done.
2. Inside WSL: install JDK 21 (section 1, step 1), clone into `~/rdt-udp`, build, and run the calibration there:

```
java -Xms512m -Xmx512m -cp build/classes emulator.Calibrate
git add results/calibration.csv
```

3. **8 pm: conventions call.** You own conventions 6, 8, 9, 10 and 11 — bring the `RunStats` field list.

Done when: `results/calibration.csv` records a WSL2 run and the ceiling is posted in the chat.

Sat 19 – Sun 20

`RunStats`; the first real run

1. Saturday: `RunStats` with the fields in convention 10, and a method that produces the `RESULT` line from convention 8. The protocols increment its counters.
2. Sunday: sit with Shaili for the first Stop-and-Wait run through your emulator. Most first-run problems are addresses and ports. Then check her `RESULT` line parses as JSON.

Done when: a Stop-and-Wait run produces one `RESULT` line that parses, with sensible numbers.

Sprint B · 21–27 September

Mon 21 – Tue 22

The runner; the decisions

1. **Mon 21:** `run_matrix.py` as described in 3.10, including derived seeds, failed rows and resuming.
2. **Tue 22:** choose the config format and lock one config per experiment in `analysis/configs/`.
3. **Tue 22:** the dry run — one seed of every Experiment 1 cell with the protocols that exist. Time the Stop-and-Wait runs and make the 8 MiB or 2 MiB decision from 3.9. Post it in the chat with the timings.
4. **Tue 22:** the JIT warm-up check from 3.7, and the **first plot** from real data.
5. **Tue 22, 10 pm:** the Selective Repeat fallback decision, with Sohini.

Done when: one command runs a sweep, a second turns the CSV into a PNG, and the file-size decision is recorded.

Wed 23 – Thu 24

Plots; help land Core complete

1. **Wed 23:** `plots.py` with the confidence intervals from 3.8 and a fixed colour per protocol across every figure. Test the interval calculation on a tiny CSV whose answer you worked out by hand.
2. **Thu 24:** confirm every configured operating point is under half the WSL2 ceiling.
3. **Thu 24:** help Sohini get Selective Repeat merged. Explain, debug and review — but her fingers type it.

Done when: a plot with error bars regenerates from a committed CSV — your part of Gate 4.

Fri 25 – Sun 27

Launch the matrix; Experiment 3

1. **Friday evening:** launch the overnight runs, Stop-and-Wait cells first. Starting on Sunday would sink the gate.
2. Run Experiment 3 from section 9.
3. **Saturday:** check the runs, and restart anything that failed — the runner resumes.
4. **Sunday:** generate Figure 3, and regenerate figures 1–3 from the committed CSVs.

Done when: figures 1–3 regenerate by script from at least 3 seeds per cell — **Gate 6, sprint done.**

After the sprint

28 Sep – 26 Oct

Buffer, then weeks 6–8

1. **28 Sep – 5 Oct:** full 10-seed runs of experiments 1–3; the methodology section with Shaili; **one trial tc netem run** so any kernel problem surfaces early.
2. **6–12 Oct:** the netem cross-validation from 3.11; `analysis/spurious.py` with Sinjan; run cross-teaching — and make sure *you* can explain Selective Repeat.
3. **13–19 Oct:** final figures; assemble the report from everyone's sections.
4. **20–26 Oct:** code freeze 22 October; confirm `make all` regenerates every figure from a clean clone inside WSL; viva drills.

6 · Team conventions — agreed on the Friday 18 call

Four people are writing code that has to plug together on Monday 21. Each item below is a decision that, left unspoken, becomes a day of integration bugs. **On Friday 18 September at 8 pm, all four of us join a 30-minute call**, confirm or change each default, and Shaili commits the result as `docs/CONVENTIONS.md` on Saturday morning. Until then, treat these as proposals — and if you disagree with one, bring the reason to the call rather than quietly doing something else.

#	TOPIC	PROPOSED DEFAULT	OWNER
1	Protocol interface	One <code>ArqProtocol</code> interface with two operations: <i>send a file to a peer</i> and <i>receive a file on a port</i> . Both take a shared <code>ProtocolConfig</code> (window, RTO mode, sequence-space bits, payload size) and return <code>RunStats</code> . The session layer — metadata handshake, FIN teardown, hash check — sits outside the protocols and wraps whichever one is chosen, so Go-Back-N and Selective Repeat never reimplement it.	Shaili
2	Sequence numbers	The handshake is a DATA packet with <code>seq = 0</code> and flag bit <code>0x01</code> (META) set. File data starts at <code>seq = 1</code> . The sequence space is k bits, set with <code>--seqbits</code> ; the default is 32 and only wraparound tests make it small. Every comparison between sequence numbers uses modular arithmetic.	Shaili
3	What an ACK number means	The <code>ack</code> field always holds a sequence number that <i>was received</i> . Stop-and-Wait and Selective Repeat acknowledge exactly that packet. Go-Back-N's <code>ack n</code> is cumulative: everything up to and including n arrived in order — the textbook convention. Each protocol restates its rule in its class comment.	Shaili, Sinjan, Sohini
4	Payload size	1400 bytes per DATA packet; the last one is shorter. An 8 MiB file is therefore 5,992 data packets plus the handshake.	Shaili
5	Command-line flags	Sender: <code>--file</code> <code>--to host:port</code> <code>--protocol stopwait gbn sr</code> <code>--window N</code> <code>--rto adaptive fixed:X</code> <code>--seqbits k</code> <code>--verbose</code> . Receiver: <code>--port</code> <code>--out</code> <code>--protocol</code> <code>--window N</code> <code>--seqbits k</code> <code>--verbose</code> . An unknown flag is an error, never silently ignored.	Shaili
6	Ports	For manual runs the emulator listens on 9000 and the receiver on 9001. The experiment harness picks fresh ports for every run.	Sohan
7	Fixed RTO	<code>--rto fixed:1.5</code> means $RTO = 1.5 \times \text{base RTT}$, where base RTT = emulator delay up + delay down . With <code>--both delay=20</code> that is 40 ms, so <code>fixed:1.5</code> is 60 ms.	Sinjan
8	Result line	At the end of a transfer the sender prints exactly one line beginning <code>RESULT</code> followed by a single JSON object of <code>RunStats</code> fields. Nothing else may print a line starting with <code>RESULT</code> . <code>run_matrix.py</code> reads only that line.	Sohan
9	What goodput means	Goodput = $\text{file bytes} \times 8 \div \text{elapsed seconds}$, where <i>elapsed</i> runs from sending <code>seq 1</code> to receiving the ACK that covers the last data packet — handshake and FIN excluded. Wire throughput = every byte the sender put on the wire, headers and retransmissions included, $\times 8 \div$ the same elapsed time.	Sohan
10	RunStats fields	<code>protocol</code> , <code>window</code> , <code>seqbits</code> , <code>rto_mode</code> , <code>file_bytes</code> , <code>elapsed_ms</code> , <code>goodput_bps</code> , <code>wire_bytes</code> , <code>throughput_bps</code> , <code>data_sent</code> , <code>retransmissions</code> , <code>timeouts</code> , <code>dup_acks</code> , <code>acks_received</code> , <code>corrupt_dropped</code> , <code>sha256_match</code> . The harness adds the run settings itself — <code>seed</code> , <code>loss</code> , <code>dup</code> , <code>corrupt</code> , <code>reorder</code> , <code>delay</code> , <code>jitter</code> .	Sohan
11	Trace lines	<code>us</code> , <code>dir</code> , <code>type</code> , <code>seq</code> , <code>ack</code> , <code>action</code> , <code>delayMs</code> . The <code>type</code> and <code>ack</code> fields are new in the emulator rewrite. Today every ACK is logged with <code>seq 0</code> , so ACKs cannot be told apart — and Experiment 4 needs to tell them apart.	Sohan
12	Logging	No printing per packet unless <code>--verbose</code> is given. Writing a line for every packet costs more than the protocol itself and makes goodput numbers meaningless.	Everyone
13	Tests and main	Every new test class is added to both <code>build.sh</code> and the <code>TESTS</code> line of the <code>Makefile</code> , or it never runs. Run <code>./build.sh</code> before every push: <code>main</code> must always build, because a compile error there stops all four of us.	Everyone

#	TOPIC	PROPOSED DEFAULT	OWNER
14	Sequence arithmetic	One small <code>SeqSpace</code> helper for every modular comparison — distance from the window base, "inside the window", "inside the previous window" — built on <code>Math.floorMod</code> and shared by Go-Back-N and Selective Repeat. The wraparound rule is then written once and tested once.	Sohini

7 · How to test your work

TEST	WHAT IT MUST PROVE
ChannelTest (14, existing)	Perfect channel untouched; same seed, identical decisions; different seeds diverge; loss does not disturb delays; measured loss and duplication rates within tolerance over 100,000 packets; duplicates arrive twice; corruption flips exactly one bit and always fails the checksum; reordering adds its penalty; jitter stays in band; delay never negative; bad specs rejected.
NetEmSmokeTest (6, existing)	Forwards in order; drops about the configured fraction; carries replies back; applies delay; delivers duplicates twice; writes a readable trace.
Trace fields (new)	An ACK's line records <code>type</code> 1 and its <code>ack</code> number; a duplicate's extra line is marked <code>dup</code> .
Direction seeds (new)	With one seed, <code>c2s</code> and <code>s2c</code> make different decisions.
RunStats (new)	The <code>RESULT</code> line is valid JSON containing every field in convention 10.
Harness (new)	<code>run_matrix.py --dry-run</code> prints the commands it would run; derived seeds are stable across runs; a timed-out run writes a failed row.
Statistics (new)	A hand-worked 3-value and 10-value example gives the right mean and interval.

Convention 13: add each new Java test class to `build.sh` and to the `Makefile`.

8 · When something breaks

SYMPTOM	LIKELY CAUSE	HOW TO CHECK
Emulator forwards nothing	<code>--to</code> points at the wrong port, the receiver is not running, or the sender is aimed straight at the receiver	Run with <code>--verbose</code>
Replies never reach the sender	NetEm has not yet seen the sender, or the receiver replies somewhere other than the packet's source	The sender must send first
"ignoring packet from unexpected source"	A sender restarted on a new port while the same NetEm kept running	Start a fresh NetEm for every run
Measured loss far above the setting	Counting <code>dup</code> lines, or loss applied on both directions and measured end to end	Count <code>c2s</code> DATA lines only
Trace ends early	Process killed before closing — at most 256 lines lost	Stop NetEm cleanly from the runner
Different seeds give identical results	Seed not passed through, or an old NetEm still on the port	Print the seed in the <code>RESULT</code> row
Goodput jumps around at 0% loss	Other programs, parallel runs, <code>/mnt/d</code> file access, or JIT warm-up	3.7
Error bars enormous	$n = 3$, where $t = 4.303$	Expected in the first pass
Everything on loopback suddenly slow	A <code>tc netem</code> qdisc left behind	<code>sudo tc qdisc del dev lo root</code>
Specified qdisc kind is unknown	<code>netem</code> module not loaded	<code>sudo modprobe sch_netem</code>
<code>bash\r: No such file or directory</code> in WSL	<code>build.sh</code> with Windows line endings	Re-clone inside WSL; <code>.gitattributes</code> now forces Unix line endings

9 · Experiment 3 — retransmissions against reordering

Question	How much does packet reordering, with no loss at all, cost each protocol in retransmissions?
Variable	Reorder probability 0, 1, 2, 5, 10, 20%
Held fixed	No loss — so every retransmission is caused by reordering; all three protocols; window 32; 20 ms delay each way; <code>reorderExtra</code> 20 ms; adaptive RTO; file size as decided on Tue 22
Seeds	At least 3 per cell by Sun 27; 10 in the buffer week
Measured	Retransmissions per 1,000 data packets, duplicate ACKs, goodput

What we expect

- **Stop-and-Wait: almost none.** One packet in flight means nothing can overtake it; a held-back packet is simply late. It retransmits only if that lateness outruns its RTO.
- **Go-Back-N: many.** The packets that overtake a held-back one arrive out of order and are discarded, so they are lost from the receiver's point of view, and the sender eventually resends the window.
- **Selective Repeat: few.** Its receiver stores the early arrivals, so it only retransmits when a held-back packet is late enough to trigger its own timer.

If a result contradicts these predictions, that is the most interesting paragraph in the report — find out why.

Figure 3

- Retransmissions per 1,000 data packets against reorder probability, one line per protocol, 95% confidence-interval error bars. The same protocol colours as figures 1 and 2.
- The caption states the settings and n .

10 · Your report section — methodology and the emulator

1. **The emulator** — the middlebox design, the impairment model and why a separate process.
2. **Reproducibility** — seeds, fixed draws and common random numbers.
3. **Calibration** — the WSL2 ceiling, the budget and the operating-point table from 3.6.
4. **Measurement hygiene** — one run at a time, warm-up, GC, recorded failures.
5. **Statistics** — confidence intervals and why n matters.
6. **Validation** — the `tc netem` comparison, differences included.
7. **Threats to validity** — one machine; no bandwidth limit; independent rather than bursty loss; the JVM.
8. **Experiment 3** — Figure 3 and its discussion.

About three pages. You also assemble the full report in week 7.

11 · How we work together

Nightly check-in — by 10 pm, in the group chat

Three lines, every night of the sprint. For example:

```
Done:      receive buffer releases 1,3,4,2 in order; 5 tests passing
Tomorrow:  duplicate handling, then open the PR
Blocked:   nothing
```

In a two-week sprint, a silent day costs whoever is waiting on you half a day. If you are blocked, say so that evening — never save it for the next gate.

Branches

Never commit directly to `main`. Use one branch per task, named after your role — `m4/short-task-name`. Always start from the latest `main`:

```
git checkout main
git pull
git checkout -b m4/short-task-name
```

Bring `main` into your branch at least once a day, so you meet other people's changes while the conflicts are still small:

```
git fetch origin
git merge origin/main
```

Pull requests

Push your branch, open a pull request on GitHub, and use this description:

```
What:      one sentence
Why:       which playbook task this is
How tested: the commands you ran and what they printed
Ask me:    two things you want the reviewer to question
```

WHEN	YOUR REVIEWER — AND WHOSE PRS YOU REVIEW
Sprint A · 14–20 September	Sohini Pandit
Sprint B · 21–27 September	Sinjan Mishra

Review within 12 hours. An unreviewed PR blocks someone. If your partner is stuck, anyone else may review instead, and you review your partner's work on the same terms.

The explanation check. Before approving, the reviewer asks two or three *why* questions — *why this data structure, what happens if this ACK is lost, why this bound and not another*. If the author cannot answer, the PR does not merge. A five-minute voice call is enough. It rehearses the viva every day, and it catches copy-pasted code while there is still time to rewrite it.

Commits

- At least **one commit every working day** of the sprint, under your own name and email.
- Small commits whose messages say what changed — *"Release buffered packets once the gap at rcv_base fills"*, not *"update"*.
- Never commit on someone else's behalf. The brief counts that as non-contribution by the absent member.
- No Co-Authored-By lines, no bot pull requests, and Dependabot stays off.** The contributor list must show exactly our four names.

Other people's files

If you need a change in a file someone else owns, message them first. If it is urgent and they are unreachable, make the smallest change you can, say so in your PR, and tag them. A merge conflict in a file is resolved by that file's owner.

12 · Using AI without losing marks

PERMITTED BY THE BRIEF

Explaining concepts · debugging · reviewing your code · generating tests · boilerplate and plotting scripts · improving your writing

NOT PERMITTED

Generating the core protocol implementation and submitting it as your own work

Our rule: use AI to understand your module, then type the code yourself. Two reasons, both about marks. The viva is individual and worth 5 of the 10 marks, and sir will ask you to change your code live — code you did not write is code you cannot change under pressure, and it scores zero however well it runs. And a chatbot asked for a textbook protocol gives every group roughly the same code. Another group in our year is doing P3, and the brief treats identical-looking submissions as collusion.

Uses that help

- "Explain Karn's algorithm with a timeline where the ACK arrives just after a retransmission." Then close it and draw the timeline yourself.
- "Here is my window check and a failing test. Why does it fail when the sequence numbers wrap?" — debugging code you wrote.
- "Write tests for a receive buffer that must release packets in order." Tests are explicitly permitted.
- "Quiz me on my module the way a viva examiner would." The single most useful thing it can do for you.

Uses that cost marks

- "Write GoBackN.java." Even if you read it afterwards, you will not be able to change it live.
- Putting AI output into a PR you cannot then explain in the review check.

Log it the same day

Whenever AI materially helped, add an entry under your role in `AI-USE.md`:

```
### 2026-09-17 – receive buffer
**What:** asked why Java's % operator goes negative for a wrapped sequence number
**Permitted under:** explaining concepts
**Mine:** the buffer design, all of the code, and the tests for the wrap case
```

Honest disclosure carries no penalty. Undisclosed use discovered at the viva does.

You are the only one of us with Claude Code. It runs inside the repository and can execute tests, which makes it very good at helping a teammate find why *their* code fails. Use it that way: explain, debug, review, write tests. Never use it to produce a teammate's module — that costs them their viva marks, and it would make your commit history look like you did everyone's work. Keep the contributor list clean too: no AI co-author lines in any commit, ever.

13 · Viva preparation

1. Why is the emulator a separate process?

- No test-only code in the protocols; works across machines; swaps for `tc netem` without a code change.

2. How do you know the same seed gives the same run?

- `java.util.Random`'s algorithm is specified; fixed draw order; the determinism test compares full decision sequences.
- Why seeds are derived from run names, not 1 to 10.

3. Why exactly ten random draws per packet?

- Common random numbers: changing loss leaves every other decision unchanged, so cells differ only in the variable. Name the test that guards it.

4. Why does the release queue need a tiebreak?

- Equal release times have no defined order in a `PriorityQueue`; without it the queue itself would reorder packets.

5. How do you know your curves measure the protocol, not your laptop running out of CPU?

- The calibration method, the WSL2 ceiling, and the operating-point table showing every run an order of magnitude below it.

6. What does the 95% confidence interval mean here? Why are the 3-seed bars so wide?

- $\bar{x} \pm t \cdot s / \sqrt{n}$; t is 4.303 for three runs and 2.262 for ten.

7. How is a retransmission labelled spurious?

- Match attempts to trace lines, skipping `dup` copies; the fate of the previous attempt; the ACK's fate via the new `type` and `ack` fields; why Selective Repeat gives exact labels.

8. How does your emulator differ from `tc netem`, and what did the comparison show?

- Uniform versus netem's jitter, default reordering, a loopback-wide effect. State the measured agreement and each difference.

9. Why corrupt exactly one bit?

- Guaranteed detectable by the checksum, so an invariant can be tested; the trade-off against realistic burst errors.

10. Live change: add a burst-loss mode.

- New config key; new draws appended *after* the existing ten; raise the draw constant; confirm the common-random-numbers test still passes.

11. Live change: make jitter normally distributed instead of uniform.

- One line in the delay calculation, still consuming one draw; clamp at zero; which tests would notice.

14 · Help, references and glossary

Who to ask

QUESTIONS ABOUT	ASK	GITHUB
Packet format, checksum, handshake, FIN, CLIs, Stop-and-Wait, the integrity suite	Shaili Seth	shailiseth24
Timers, RTT estimation, Karn's algorithm, Go-Back-N, fixed and adaptive RTO	Sinjan Mishra	Thesin-01
Selective Repeat, the receive buffer, window bounds, wraparound	Sohini Pandit	sohinix
The emulator, trace files, calibration, RunStats, CSVs, plots, repository access	Sohan Mandal	sohan1611

Stuck for more than an hour? Say so in the group chat. An hour stuck in a two-week sprint is expensive; two minutes of someone else's attention usually is not.

References

- **RFC 1071**, *Computing the Internet Checksum* — section 1 is the algorithm.
- **RFC 6298**, *Computing TCP's Retransmission Timer* — Jacobson/Karels estimation, Karn's algorithm, backoff.
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach* — section 3.4 (reliable data transfer: rdt 3.0, Go-Back-N, Selective Repeat) and section 3.5 (TCP round-trip time estimation).
- **Java 21 API documentation** — [java.nio.ByteBuffer](#), [java.net.DatagramSocket](#), [java.util.PriorityQueue](#), [java.security.MessageDigest](#), [java.util.Random](#), [Math.floorMod](#).
- **In the repository** — [docs/PLAN.md](#) for the design decisions, [docs/member-playbook.html](#) for everyone's dates, [AI-USE.md](#), and the existing tests, which show the house test style.

Glossary

ARQ	Automatic Repeat reQuest: reliability by detecting loss and retransmitting. Stop-and-Wait, Go-Back-N and Selective Repeat are three ARQ schemes.
RTT	Round-trip time — data out, ACK back.
RTO	Retransmission timeout — how long the sender waits for an ACK before resending.
SRTT, RTTVAR	The smoothed RTT and its smoothed variation, from which RTO is computed (RFC 6298).
Cumulative ACK	"Everything up to n has arrived." Used by Go-Back-N.
Selective ACK	"This particular packet has arrived." Used by Selective Repeat.
Window	How many unacknowledged packets the sender may have in flight at once.
Sequence space	The range of sequence numbers: 2^k values for k bits, wrapping back to 0.
Goodput	Useful file bytes delivered per second — excluding headers and retransmissions.
Throughput	Every byte put on the wire per second — including headers and retransmissions.
Spurious retransmission	Resending a packet whose original actually arrived.
Network byte order	Big-endian: most significant byte first. ByteBuffer 's default.
One's complement sum	Addition where a carry out of the top bit is added back in at the bottom — the end-around carry. The basis of the Internet checksum.
Seed	The starting value for the emulator's random generator. Same seed, same decisions, on any machine.
95% confidence interval	A range around a measured mean reflecting run-to-run variation. Every data point we plot carries one.
JIT warm-up	The JVM runs code slowly at first and compiles hot paths as it goes, so the first packets of a run can be slower.

15 · Sprint checklist

- ☐ **Mon 14** — `ChannelConfig` and `TraceLog` rewritten; `type` and `ack` in trace lines
- ☐ **Wed 16** — `Channel` rewritten; `ChannelTest` 14/14
- ☐ **Thu 17** — `NetEm` rewritten; `NetEmSmokeTest` 6/6; merged
- ☐ **Fri 18** — `Calibrate` rewritten; `AI-USE.md` box ticked; JDK in WSL; WSL2 ceiling committed; 8 pm call
- ☐ **Sat 19** — `RunStats` and the `RESULT` line
- ☐ **Sun 20** — first Stop-and-Wait run through your emulator parses
- ☐ **Mon 21** — `run_matrix.py` with derived seeds, failed rows and resume
- ☐ **Tue 22** — configs locked; dry run timed; file size decided; warm-up checked; first plot; fallback call
- ☐ **Wed 23** — `plots.py` with confidence intervals
- ☐ **Thu 24** — operating points checked; helped land Core complete
- ☐ **Fri 25** — overnight runs launched
- ☐ **Sun 27** — Figure 3; figures 1–3 regenerate by script
- ☐ **Every night** — three-line check-in by 10 pm; at least one commit every working day